05 Manual de instalacion SaborExpress

Equipo de desarrollo SaborExpress

Filiación institucional y autores individuales: por confirmar

Estudio académico de ingeniería de software y documentación

14 de octubre de 2026

Manual de instalación y configuración

Proyecto: SaborExpress | **Fecha de corte:** 14 de octubre de 2026 | **Versión:** 2.0 | **Tipo:** Documento técnico y académico

Estado de la evidencia: inspección documental y estática del código. Las pruebas de ejecución y los datos institucionales no aportados no se presumen realizados.

1. Requisitos mínimos de entorno de desarrollo

Windows, macOS o Linux compatible, Git, .NET SDK 8, Node.js LTS compatible con Next.js 16 y Expo 57, npm, acceso a un gestor relacional soportado por la configuración .NET y una cuenta de pruebas para las integraciones externas. La capacidad de RAM/CPU y versiones exactas de emuladores se deben ajustar y medir según hardware real; no se promete un mínimo productivo universal.

2. Descarga y ramas

mkdir C:\Dev\SaborExpress -Force

cd C:\Dev\SaborExpress

git clone https://github.com/yersonct/SaborExpressBack.git back

git clone https://github.com/yersonct/SaborExpressWeb.git web

git clone https://github.com/yersonct/SaborExpressMovil.git movil

git clone https://github.com/yersonct/SaborExpressDocs.git docs

La ubicación es un ejemplo. Registrar ramas y SHA con git -C .\back rev-parse HEAD y comandos equivalentes. No dar por supuesto que todas las dependencias están sincronizadas en main.

3. Backend ASP.NET Core

```
cd C:\Dev\SaborExpress\back
dotnet --info
dotnet restore
dotnet build -c Debug
```

Configuración: DatabaseProvider acepta literalmente MySQL, SqlServer o PostgreSQL según DatabaseConfiguration.cs. El ConnectionStrings debe incluir, según el proveedor, MySQLConnection, SqlServerConnection o PostgresConnection. Los valores sensibles deben ir en variables de entorno o dotnet user-secrets; no copiar cadenas reales del appsettings.json al documento. Revisar JwtSettings, AllowedOrigins, correos, Wompi y cualquier clave externa en clases de configuración.

Ejemplo sintáctico sin credenciales:

```
dotnet user-secrets init
dotnet user-secrets set 'DatabaseProvider' 'PostgreSQL'
dotnet user-secrets set 'ConnectionStrings:PostgresConnection'
'<CADENA_LOCAL_DE_PRUEBA>'
```

La API llama migraciones y seeders al arrancar. **Primero usar una BD desechable respaldada**, porque MigrateAsync puede alterar el esquema. Lanzar dotnet run solo después

de validar los secretos y confirmar el proveedor. Swagger UI aparece en la ruta / cuando el proceso y el entorno permiten acceder a ella. No se garantiza URL o puerto fijo.

4. Web Next.js

```
cd C:\Dev\SaborExpress\web
npm ci
npm run dev
```

npm run build permite comprobar empaquetado. Ajustar la URL base de la API según variables reales usadas en src/config o servicios; no asumir un nombre de .env no comprobado. La URL que imprima Next.js será el punto de acceso local.

5. Móvil Expo

```
cd C:\Dev\SaborExpress\movil
npm ci
npx expo start
```

En un teléfono físico se necesita dirección IP accesible por red, no localhost del PC. Expo 57 usa dev client o flujo admitido por la versión; confirmar compatibilidad de Expo Go/dispositivo, permisos de ubicación y push. Registrar token push solo en ambiente de prueba autorizado.

6. Wompi, SignalR y servicios externos

Usar ambiente sandbox y llaves de prueba; validar firmas de eventos mediante el código del servicio. No activar pagos reales en pruebas. Para SignalR probar sesión JWT, negociación, reconexión y comportamiento cuando no hay conexión.

7. Verificación y resolución de fallas

Problema	Comprobación	Solución segura
dotnet restore falla	SDK 8 y paquetes	Revisar feeds/proxy sin exponer credenciales
No conecta BD	Provider/nombre de cadena/servicio	Probar conexión local y permisos
Migración falla	Proveedor e historial	Restaurar BD de pruebas y revisar script
CORS rechaza	Origen web, URL API	Ajustar AllowedOrigins para ambiente
Móvil no conecta	IP/firewall/HTTPS	Usar IP de PC accesible, TLS de prueba
Wompi no confirma	Referencia y firma	Inspeccionar sandbox sin reintentos reales
Web no compila	Node/npm/dependencias	npm ci y revisar errores exactos

8. Desinstalación y respaldo

Detener procesos; respaldar BD y archivos según política; eliminar instalaciones de prueba después de garantizar recuperabilidad. Nunca ejecutar eliminación de volúmenes o base productiva como procedimiento genérico.

8. Instalación reproducible por componente

Etapa	Comando o actividad	Resultado esperado
Comprobar herramientas	dotnet --info; node --version; npm --version; git --version	Revisar versión SDK .NET 8, Node compatible con Next.js 16, Git y herramientas móviles.
Obtener repositorios	git clone https://github.com/yersonct/SaborExpressBack.git y sus equivalentes Web/Movil	Guardar SHA reproducible con git rev-parse HEAD.
Restaurar API	cd SaborExpressBack; dotnet restore SaborExpress.csproj	Sin requerir conexión a BD para restaurar paquetes.
Compilar API	dotnet build SaborExpress.csproj -c Release	Compilación sin errores; no equivale a prueba funcional.
Preparar base de datos	crear BD/usuario con privilegios mínimos e inyectar ConnectionStrings	Proveedor oficial de desarrollo debe coincidir con DatabaseProvider.
Ejecutar API	dotnet run --project	Consultar URL y puerto de consola,

Etapa	Comando o actividad	Resultado esperado
	SaborExpress.csproj	abrir Swagger en raíz si configuración lo permite.
Restaurar web	cd SaborExpressWeb; npm ci	Dependencias iguales a package-lock.
Compilar web	npm run build	Validar variables de API y route protection.
Iniciar web	npm run dev	Url mostrada por consola; por defecto Next suele utilizar localhost:3000, verificar.
Restaurar móvil	cd SaborExpressMovil; npm ci	Acordar Node/SDK con versiones Expo declaradas.
Iniciar móvil	npm start o npx expo start	Elegir emulador/dispositivo según SDK requerido; URL API accesible.
Verificar funcionalidad	inspeccionar Swagger, crear cliente sintético y consultar catálogo	No usar datos ni pagos de producción en prueba.

9. Configuración de ambiente de forma segura

API: seleccionar proveedor DatabaseProvider (MySQL, SqlServer o PostgreSQL) y la cadena correspondiente (MySQLConnection, SqlServerConnection, PostgresConnection). Revisar además configuración JWT, orígenes permitidos CORS, correo, geocodificación y Wompi antes de levantar el proceso. Los nombres de claves secundarios deben derivarse de appsettings.json y settings de clases, sin inventar valores concretos.

Secretos: almacenar credenciales mediante User Secrets de .NET o variables del orquestador en desarrollo, y gestor de secretos en producción. No reutilizar appsettings.json sensible, google-services.json ni credenciales cargadas por compañeros como ejemplos públicos.

Web: identificar variable que configura la URL de API leyendo src/config y servicios antes de modificar. Next.js expone al navegador variables con NEXT_PUBLIC_: nunca colocar claves privadas en ellas.

Móvil: para teléfono físico, localhost apunta al teléfono y no a la PC. Usar URL accesible en la red de desarrollo, TLS apropiado y evitar tunnels públicos con información real.

Expo Go puede carecer de módulos nativos que exijan development build; verificar condiciones de Expo 57.

Datos de semilla: Program.cs puede aplicar migraciones y seeders al arrancar. Antes de la primera ejecución, comprobar que el seeder inicial no incluya una contraseña fija en código o configuración distribuida.

10. Procedimiento Windows PowerShell (ejemplo reproducible)

```
# En un directorio de trabajo seguro, con Git, .NET 8 y Node instalados
git clone https://github.com/yersonct/SaborExpressBack.git
git clone https://github.com/yersonct/SaborExpressWeb.git
git clone https://github.com/yersonct/SaborExpressMovil.git

cd SaborExpressBack
git rev-parse HEAD
dotnet restore .
dotnet build . -c Release
# Configurar cadena de conexión y secretos ANTES de ejecutar
# dotnet run --project SaborExpress.csproj

cd ..\SaborExpressWeb
npm ci
npm run build
```

```
# npm run dev

cd ..\SaborExpressMovil
npm ci
# npm start
```

Los comandos de ejecución se dejan comentados por seguridad hasta configurar un motor de datos de prueba y variables apropiadas. El SDK y los puertos definitivos se documentan después de instalar en el equipo destino.

11. Preparación de despliegue y puesta en marcha

Preflight: comprobar compatibilidad de SO y arquitectura, IP/dominio, puertos, proxy y TLS, archivos de configuración, migraciones pendientes, estado de dependencias, permisos del usuario de DB, espacio disponible, sincronización horaria y acceso Wompi sandbox.

Paso controlado: generar backup → compilar backend y web con SHA aprobado → exportar migración revisada → aplicar en staging → ejecutar pruebas humo de Auth/Products/Orders/Payments → habilitar entorno → monitorizar y conservar artefactos del despliegue.

Plan de reversión: mantener binarios previos, export de esquema y respaldo. Ante migración no reversible, restaurar BD en nueva instancia validada antes de apuntar clientes; no usar `database update 0` ni revertir migraciones indiscriminadamente sobre producción.

Separación de ambientes: DEV usa datos sintéticos y Wompi sandbox; STAGING replica configuración sin datos personales; PROD necesita secretos productivos, TLS real, backups, monitoreo y procedimientos firmados. No mezclar URL ni clave Wompi de test/prod.

12. Diagnóstico de instalación

Error	Causa probable	Validación	Solución prudente
dotnet restore falla	red o versión SDK incompatible	dotnet –info; NuGet	revisar SDK net8.0
Migration failed	cadena incorrecta, falta DB o permisos	logs sin secretos; provider EF	reparar BD, repetir en sandbox
next build falla	Node o variable obligatoria	npm run build, package.json	alinear versión y dependencias
app móvil no conecta	localhost erróneo o red bloqueada	ping y URL accesible desde teléfono	usar IP red o túnel protegido
401 login	JWT/config de usuario	logs sanitizados y claims	revisar Auth/seeders
WebSocket no conecta	proxy, CORS o credenciales	Network tab y servidor	proxy websocket/cors seguros
Webhook no llega	HTTPS o URL mal configurada	panel Wompi sandbox	URL pública HTTPS solo sandbox

13. Lista de aceptación de instalación

- ☐ API compila y publica su versión y SHA.
- ☐ DB seleccionada soporta migraciones nuevas en base efímera.
- ☐ Swagger accesible únicamente conforme al ambiente y su política.
- ☐ Web compila y consulta API de prueba.
- ☐ Móvil permite una sesión de prueba en el entorno objetivo.
- ☐ Inicio/cierre de sesión y lectura del catálogo probados.
- ☐ Datos de prueba aislados de producción.
- ☐ Firma Wompi sandbox y evento duplicado ensayados.
- ☐ Copia/restauración de DB simulada.
- ☐ Responsable firma acta, versión y evidencias.

Estado de esta lista: no ejecutada; debe diligenciarla el responsable que despliegue.

Fuentes primarias de implementación

- API: https://github.com/yersonct/SaborExpressBack/tree/main
- Web: https://github.com/yersonct/SaborExpressWeb/tree/main
- Móvil: https://github.com/yersonct/SaborExpressMovil/tree/main
- Referencia de presentación: https://github.com/jsgh1/SRD-v1/tree/dev/docs/docs-SRD

Control documental: documento elaborado a partir del estado observado en octubre de 2026. Antes de entregar, verificar el commit exacto, responsable, revisión del instructor y cualquier cambio posterior del código.